

Common Data Store Library User Guide

V-2023.09

May 2023

Email your comments about this manual to:
cdpl-dev@synopsys.com.



Copyright Notice and Proprietary Information

© 2023 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

TABLE OF CONTENTS

CDSL USAGE	5
INTRODUCTION	5
OPERATIONAL ENVIRONMENT	5
DATA STORE SERVER	7
STARTUP AND SHUTDOWN	7
SERVER MANAGEMENT USING CDPL	7
DEFAULT CONFIGURATION	8
REMOTE AND POOL SERVER CONFIGURATION	8
CDSL IN-MEMORY DATA STORE SERVER	9
COMMAND LINE OPTIONS	9
DATA SHARDING	10
BACKGROUND PERSISTENCE	11
CDSL ON-DISK DATA STORE SERVER	13
COMMAND LINE OPTIONS	13
HOSTS INFORMATION FILE	14
DEFAULT HOSTS INFORMATION FILE	14
HOSTS INFORMATION FILE FORMAT	14
USING SH TO START THE DATA STORE SERVER	15
USING RSH TO START THE DATA STORE SERVER	15
USING SSH TO START THE DATA STORE SERVER	16
WORKER INFORMATION FILE	17
DEFAULT WORKER INFORMATION FILE	17
WORKER INFORMATION FILE FORMAT	17
S3 COMPATIBLE STORAGE SUPPORT	19
CONNECTING TO SERVER	19
DEFAULT BUCKET	19
SHARED LIBRARY	20
DEBUGGING DATA STORE ENVIRONMENTS	21
LOG FILES	21
<i>Server Log Files</i>	21
<i>Client Log Files</i>	22

<i>CDSL Version</i>	<i>22</i>
LOG VIEWING UTILITIES: LOG2TTY AND LOG2HTML	23
<i>Sample server log from a successful run</i>	<i>23</i>
<i>Sample server log with errors</i>	<i>24</i>
DPCHECK – INFRASTRUCTURE VALIDATION FOR THE DATA STORE SERVER	24
<i>Checking a Host Information file with dpcheck</i>	<i>26</i>

1

CDSL Usage

Introduction

CDSL (Common Data Store Library) is a proprietary software library used by Synopsys EDA products. CDSL offers scalable datastore **API** and run-time session management built on top of open-source datastores or cloud storage systems, allowing storing and sharing of data among distributed computation nodes. Data persistence is also possible.

Operational Environment

When integrated with Synopsys EDA applications, CDSL runs behind the scenes and offers services for distributed processing. CDSL service setup and configuration are expected to be done and/or can be done through an EDA product's configuration parameters.

Use the following CDSL environment variables to set up and configure distributed data storage and data retrieval. Most applications set these variables internally and do not require users to manually set them.

CDSL_CONFIG_MAX_KVSIZE: <OPTIONAL>: The maximum size in bytes that is allowed for a single key-value pair.

CDSL_CHECKSUM_DISABLE: <OPTIONAL>: Set this variable to 1 (one) to disable checksum calculation associated with each key-value pair stored.

CDSL_HOME: <OPTIONAL>: The root for the location of CDSL-related executables and libraries. \$CDSL_HOME/bin is one of the default directories that is searched when starting the data store server.

CDSL_IO_THREADS: <OPTIONAL>: CDSL datastore server (in memory) by default uses a single thread for I/O. Setting this variable with a value larger than one allows the CDSL server to use multiple threads for I/O.

CDSL_LOGDIR: <OPTIONAL>: The location where CDSL log files are written. If not specified, it defaults to the current working directory.

CDSL_LOG_LEVEL: <OPTIONAL>: Controls the logging of messages for clients of the data store server. Set it to 0 (that is, the lowest log level) such that clients can create log files and log debug messages. No default value exists for this variable.

CDSL_SERVER: <OPTIONAL>: The host: port address of the data store server. This is set by default in an EDA application when it initializes CDSL. If the EDA application uses CDPL (Common Distributed Processing Library) for distributed processing, this variable is automatically passed to DP workers. If the EDA application does not use CDPL for distributed processing, the application is responsible for passing this value to DP workers to give them access to the data store server.

CDSL_SERVER_LAUNCH_TIMEOUT: <OPTIONAL>: This variable sets a timeout value that overrides the default timeout value. The default value at the time of the data store server startup, that is, the launching of the data store or during the launch is 120 seconds. Note the following behaviors for this variable:

0 - CDSL falls back to its default timeout.

-1 - CDSL will not timeout data store server(s) startup.

CDSL_SERVER_IPADDR: <OPTIONAL>: Specify the IP address from where the redis server will be run. This is useful when a machine has a multi-homed interface, and a specific one from the multiple interfaces can be used. This can also be achieved with the “*bind*” field in the configuration file. If both the “*bind*” field and CDSL_SERVER_IPADDR are set, then CDSL_SERVER_IPADDR will take precedence.

CDSL_SUMMARY_SAMPLING_INTERVAL: <OPTIONAL>: This variable defines the data collection frequency of the CDSL server when it provides performance summary data. Set this variable in seconds. If not defined or set to 0 (that is, zero), CDSL server does not provide performance summary.

2

Data Store Server

CDSL's Data Store server may run in a few modes: in-memory, on-disk, or in-memory with data persistence on object data stores.

Startup and shutdown

Through CDSL's server initialization API, an EDA application brings up a Data Store server. Through CDSL's server shutdown API, the EDA application shuts down the data store server. These are all done automatically without end users' involvement.

Server Management Using CDPL

CDSL uses Synopsys CDPL (Common Distributed Processing Library) to manage the data store server.

You can configure the Data Store server through CDPL-like hosts and worker information files. For example, users or administrators can configure the server operation to specify the number of server instances that will run, the systems they will run on, and the methods (that is, launch protocol) used to start them.

Default Configuration

The default configuration of the data store server is a single instance running on the local system. If configuration files are not present, the data store server will operate in the single instance mode.

Remote and Pool Server Configuration

Alternately, you can configure the Data Store server to run on a remote system using RSH or SSH protocol. If necessary, you can configure a pool of servers to work together to share the workload and data storage.

You can create CDPL-like hosts and worker information files to configure the Data Store server to run on a remote system, or to configure a pool of servers. These simple text files are described in the following sections.

- [Hosts Information File](#) – configure the server to run on a remote system.
- [Worker Information File](#) – configure the number of servers to run in a pool.

In a normal pool server configuration, you can use both configuration files to have the pool of servers run on different systems.

3

CDSL In-Memory Data Store Server

The CDSL In-Memory Data Store server is built on a modified version of the open source `redis`, with additional (optional) supports for persistent data storage on object data stores.

The CDSL In-Memory Data Store server executable file is known as **Rediss**. In the following sections, the term “Rediss” will be used to refer to the CDSL In-Memory data store server.

Command Line Options

By default, Rediss is started with no command line options. Rediss supports several command-line options to enable features such as disk backup and restore, log message level, and port assignment.

Note:

1. The typical CDSL users are not expected to explicitly use these Rediss command line options, and this information below is provided only for debugging, that is, when you review the CDSL logs.
2. There is no guarantee that CDSL rediss is fully compatible with the Rediss executable that is built out of the open source `redis`.

Rediss supports the following command line options:

`-port (-p) port_number` where you can specify the port number to use.

By default, the server assigns itself a valid port. Use this option to assign the server a specific port.

`-dump (-d) [filename]` where *filename* is the optional destination file.

This option sets default saving options, both periodic and on exit. If the *filename* is not provided, the destination file is **dump.W\$CDPL_WORKERID.rdb**. The variable `$CDPL_WORKERID` will be replaced by the server id.

`-load (-l) [filename]` where *filename* is the optional data file.

This option specifies the data file to load when the server starts. If *filename* is not found or an error is encountered during loading, then Rediss will exit with an error.

If the *filename* is not provided, the data file is **dump.W\$CDPL_WORKERID.rdb**. The variable `$CDPL_WORKERID` will be replaced by the server id.

`-config (-c) filename` where *filename* contains configuration information.

Any variables set here will override the settings on the command line.

Only the variables you wish to set need to be set in the configuration file.

`-loglevel (-v) nolog | warning | notice | verbose | debug`

This specifies the level of logging info to store.

nolog: No logging at all

warning: Only critical errors

notice: Only important notices (recommended production setting)

verbose: Important & not so important notices (CDSL default)

debug: Everything (for development use only)

`-help (-h)` prints the usage information.

This is useful when you call Rediss directly from the command line.

Data Sharding

For key values greater than 4GB, Rediss internally chops up the data into smaller chunks and stores it in the data store, and then reassembles it to the original data when retrieving it. Specifically, the smaller chunks are stored as key values of size 3GB by default. Thus, for example, a 5GB key will be stored as a 3GB and 2GB chunk internally.

You can override this default 3GB chunk size by setting the `CDSL_DEBUG_CHUNK_SIZE` environment variable to values between

"1073741824"(1GB) and "3221225472"(3GB) on the client. For example, set this variable to 1073741824 to make the smaller chunks of size 1GB. So, the 5GB key will be stored as 5 chunks of 1GB each.

While you can potentially store large amounts of data in a single key, the recommended size is around 10GB for performance reasons.

In the pool mode of operation, you have the option to store all chunks on a single server or distribute all chunks evenly among all available servers with a session configuration parameter, `CdslSessionConfig::se_group_shards`. Specifically, if this parameter is set to 1, all shards of the key will be stored on a single server. By default, the parameter is set to 0, which means CDSL distributes the shards among all the available servers.

To disable the data sharding feature, set the CDSL environment variable `CDSL_DISABLE_SHARDING` to 1 on the client side.

Background Persistence

Rediss supports a configuration option "`ds_persist_type`" to initialize a persistent data store in the background. The persistent data store will start on the machine with the main Rediss server.

Option 1 - Leveldb: Set `ds_persist_type = CDSL_PERSIST_DISK` to launch Leveldb as a persistent store in the background. Provide the path for writing data with the config parameter "`ds_storage_dir`". The replicated data is stored at this location. Starting a new server using the same path gives you access to all the previously stored data.

Option 2 - S3 server: Set `ds_persist_type = CDSL_PERSIST_S3` to connect to the S3 compatible server for persistent store in the background. You need to provide the following server information (via `CdslPersistServerConfig`):

- Server IP (required)
- Server port (required)
- AWS_ACCESS_KEY_ID (required)
- AWS_SECRET_ACCESS_KEY (required)
- Default bucket name (required)

To use this feature, confirm that all the needed CDSL shared libraries are present. Specifically, when installed at the customer site, install the shared libraries under `$CDSL_HOME/lib/$TARGET_ARCH/`.

4

CDSL On-Disk Data Store Server

The CDSL On-Disk Data Store server is built on LevelDB, a disk-backed key-value data store server.

The CDSL On-Disk Data Store server executable file is known as **snpsds**. In the following sections, the term “Snpsds” is used to refer to the CDSL’s On-Disk Data Store server.

Command Line Options

By default, Snpsds is started with 2 command line options: `-type` and `-name`.

Snpsds supports the following command line options:

- `-p port_number` where you can specify the port number to use.
By default, the server assigns itself a valid port. Use this option to assign the server a specific port.
- `-name filename` contains the path to the Snpsds storage directory.
- `-type type_name` is the data store type name. Currently, the data store *type_name* can only be **leveldb**.

5

Hosts Information File

The Hosts Information file allows the EDA application user or administrator to specify the Data Store server host machines and the protocol for launching the server on remote hosts.

Default Hosts Information File

The CDSL initialization process by default will search for a hosts information file named “**syncdsl.hosts**”. If this file is present, the Data Store server will be started using the configuration specified in it.

An EDA application may provide its method of specifying a hosts information file for the Data Store server.

Hosts Information File Format

The hosts information file is an ASCII file. Single line comments are allowed and must be prefixed with ‘#’. Every line in the file has the following format:

<flag>| <hostname>| <slots>|<tmpDir>|<protocol>|<command>

Note that all fields except the last field are delimited with ‘|’.

Field	Value	Purpose
<i>Flag</i>	Integer – can be ‘0’ or ‘1’	A ‘0’ signifies that the host cannot be used, and a ‘1’ denotes a usable host machine for the Data Store server
<i>Hostname</i>	String, valid hostname	This is the hostname of a valid Data Store server machine. For SH protocol, this field is ignored. SH will only use the local machine.

<i>Slots</i>	Integer – any valid integer	This number is used to denote the number of slots available on the host for server instances.
<i>Temporary directory</i>	String – a valid, writable directory	At present, this field is not used and can be left empty.
<i>Protocol</i>	String identifier	This string ID signifies the command-mode used to connect to the server host machine. Valid protocols are SH, RSH, and SSH.
<i>Command</i>	String – the connection command	The given string denotes the actual string used to connect to the server machine. Use the appropriate command that corresponds to the protocol you select. Valid combinations are “SH” in field 5 with “sh” in field 6, “RSH” in field 5 with “rsh” in field 6, and “SSH” in field 5 with “ssh” in field 6.

Using SH to start the Data Store Server

```
# HOSTS INFORMATION FILE (SH)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol | command
#
1|localhost| 2|  | SH| sh
```

This hosts information file specifies a 2-server pool with both server instances on the localhost. When using the SH protocol, the *hostname* field is not required to be set because all the workers run on the localhost. Here, we have set hostname to “localhost” as a reminder.

Using RSH to start the Data Store Server

```
# HOSTS INFORMATION FILE (RSH)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol | command
#
1|teralab-x15| 1|  | RSH| rsh
1|teralab-x16| 1|  | RSH| rsh
```

This hosts information file specifies a pool of Data Store servers with one instance on teralab-x15 and one instance on teralab-x16. The servers will use RSH.

Using SSH to start the Data Store Server

```
# HOSTS INFORMATION FILE (SSH)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots| tmpDir| protocol | command
#
1|syn_lab-5| 1| | SSH | ssh
1|syn_lab-6| 1| | SSH | ssh
1|syn_lab-7| 1| | SSH | ssh
```

The above hosts information file specifies a 3-server pool with one server instance each on syn_lab-5, syn_lab-6, and syn_lab-7. You can use SSH to start the servers.

SSH requires password-less login. The following are the steps to set up password-less login for SSH.

```
% cd ~/.ssh
% ssh-keygen -f id_rsa -N '' -t rsa -q
[You do not need a pass phrase.]
% cp id_rsa.pub authorized_keys
```

6

Worker Information File

The Worker Information file allows the EDA application user to specify information about the data store server. This information includes the following:

- The type of Data Store server required. This is an integer number.
- The number of instances of the server.
- The server executable name and location.
- The server command line options.

Default Worker Information File

By default, the CDSL initialization process will search for a worker information file named “**syncdsl.workers**”. If the file is present, the Data Store server will use the configuration specified in the file.

An EDA application may provide its method of specifying a worker information file for the data store server.

Worker Information File Format

The Worker Information file is an ASCII file. Single line comments are allowed and must be prefixed with ‘#’. Every line in the file has the following format.

```
<worker_type>; <max_workers>; <min_workers>; <server_executable and  
parameters>
```

Field	Value	Purpose
<i>Server type</i>	Integer	The type of data store server. Set this number to 100000 for the In-Memory (Rediss) Data Store server.
<i>Max servers</i>	Integer	The maximum number of server instances. CDSL will attempt to start the maximum number of server instances.
<i>Min servers</i>	Integer	The minimum number of Data Store servers needed for successful operation. It should be at least '1'.
<i>Server executable</i>	String – a valid executable program or script	The full path of the server executable. The executable for the In-Memory data store server is “rediss”. Environment variables can be used to specify the server path. For example, {CDPL_HOME}/bin uses the environment variable “CDPL_HOME” to specify the server directory path.
<i>parameters</i>	String	The server command line parameters. The server executable and parameters form the command line that will be executed.

Here is a sample worker information file.

```
# Worker Information file
#
100000; 3; 1; {CDPL_HOME}/bin/rediss -dump save.W$CDPL_WORKERID.rdb
```

In the above example, three instances of the In-Memory Data Store server will start. Rediss command line option “-dump” specifies that Rediss saves its data to a file before exiting. When generating the output files, the special variable “\$CDPL_WORKERID” is replaced by the server instance number. Thus, you can expect that the files *save.W1.rdb*, *save.W2.rdb*, and *save.W3.rdb* will be output from this run.

Note: The command-line options set in the worker information file overrides any options that are by default set in an EDA application.

7

S3 Compatible Storage Support

S3 (Simple Storage Service) compatible storage is a data storage solution that allows access to and management of the data it stores over an S3 compliant interface. Our solution enables you to configure/connect to any S3-compatible storage server (for example, AWS S3, MinIO).

Connecting to Server

To connect to an S3-compatible server, provide the following information (via `CdslS3SessionConfig`).

- Server IP (required)
- Server port (required)
- `AWS_ACCESS_KEY_ID` (required)
- `AWS_SECRET_ACCESS_KEY` (required)
- Region (optional)
- Scheme (Optional)
- Default bucket name (optional)

Default Bucket

For simple applications that need only one bucket, CDSL creates a default bucket so that all common operations (put, get, has, delete) can proceed with the default bucket.

The default bucket name is in `${HOST}.${UserID}.${ProcessID}` format, but you can specify the default bucket name and pass that information to CDSL through an EDA application's configuration and/or control parameter(s).

Shared Library

CDSL's S3 client library is built and installed as `libcddl_s3.so`. When installed at the customer site, install the shared library under

`$CDSL_HOME/lib/$TARGET_ARCH/`.

The CDSL S3 client library depends on some shared libraries built from AWS SDK. Install those shared libraries also under `$CDSL_HOME/lib/$TARGET_ARCH/` on the customer site.

8

Debugging Data Store Environments

Distributed processing environments often depend on the customer configuration of their infrastructure.

To aid field engineers CDSL provides the tools and utilities to capture and log detailed runtime information. It also provides a method to analyze and validate the Data Server startup environment.

Log Files

CDSL uses the same logging interface as CDPL to log runtime information. Because the log files contain lots of detailed runtime information, it will be easier to view them through one of the [log viewing utilities](#): log2tty or log2html.

Server Log Files

The data store server will generate two or more log files when used in an application. These logs are informative and useful when investigating problems with the Data Store during a distributed processing run.

The data store server creates the following log files:

```
dsmaster.<hostname>.<pid>.log  
dsmaster.<hostname>.<pid>.err
```

```
dsserver.W#.<hostname>.<pid>.log
```

```
...
```

Logs having the `dsmaster` prefix are from the CDSL server launching program. If any error occurs at the farm interface level, error messages are stored in the `dsmaster.*.err` file.

Logs having the `dsserver` prefix are from CDSL's in-memory datastore servers as DP workers. The number of worker log files equals the number of Data Store server instances used. In the default configuration, there will be a `dsmaster` log file and a `dsserver` log file.

The `dsmaster` log file is most informative on the Data Store server, though no EDA-specific details are logged. Always check the `dsmaster` log first when investigating infrastructure problems.

If LevelDB is used for on-disk datastore or data persistence, its log file will be a DP worker log in the following form:

`snpsds.W#.<hostname>.<pid>.log`

Client Log Files

Client log files are disabled by default. This is because they log every transactional information (not EDA-specific data) from a client to the Data Store server and thus can be extremely verbose. If you wish to enable client logging, set the environment variable `CDSL_LOG_LEVEL` to 0.

When client logging is enabled, the log files are as shown below.

`cdsl_cli.<hostname>.<pid>.log`

There will be one log file for each client of the data store server.

CDSL Version

CDSL shares the same release version label as that of CDPL. You can find that version string in a log file's header section. For example,

```
@@start_of_log@@
time: 2020/05/22 11:39:49 PDT -0700
```

```
version: Q 2019.12 SP1 PROD CDPL
system: Linux 2.6.32-754.11.1.el6.x86_64 #1 SMP Tue Feb 26 15:38:56
UTC 2019
...
```

Log Viewing Utilities: log2tty and log2html

These utilities filter the CDSL log file contents and produce the outputs in a more readable format. Use log2tty when viewing the output in a command shell window. Use log2html to generate output for viewing in a web browser. The typical usage is shown below.

log2tty <filename>

log2html <filename> > <filename.html>

where <filename> is a CDSL log file.

Sample Server Log from a Successful Run

Here is an example of the output from log2tty when viewing a server log file from a successful DP run that uses the Data Store server. The highlighted line shows that the data store server was started successfully.

```
sh-3.00$ log2tty master.*.log | more
PDT 2013/07/31 16:43:05.107116 -- time: 2013/07/31 16:43:05 PDT -0700
PDT 2013/07/31 16:43:05.107125 -- version: CDPL_I-2013.09_PROD
PDT 2013/07/31 16:43:05.107130 -- system: Linux 2.6.18-194.8.1.el5 #1 SMP Wed Jun 23 10:52:51 ED
T 2010
PDT 2013/07/31 16:43:05.107134 -- processor: x86_64 64-bits
PDT 2013/07/31 16:43:05.107137 -- hostname: varuna010
PDT 2013/07/31 16:43:05.107160 -- new server S830161192
PDT 2013/07/31 16:43:05.108589 -- new master M1
PDT 2013/07/31 16:43:05.108607 -- bcast dir is `/remote/t3dev1/geiszler/cdsltest/cdpl/cdsl/unit/
ds_basic'
PDT 2013/07/31 16:43:05.109942 -- creating SH farm on localhost
PDT 2013/07/31 16:43:05.109960 -- exporting CDPL_LOGDIR as `/remote/t3dev1/geiszler/cdsltest/cdp
l/cdsl/unit/ds_basic'
PDT 2013/07/31 16:43:05.109966 -- SNPSLMD_LICENSE_FILE is not set
PDT 2013/07/31 16:43:05.109972 -- exporting CDSL_WORKDIR as `/remote/t3dev1/geiszler/cdsltest/cd
pl/cdsl/unit/ds_basic'
PDT 2013/07/31 16:43:05.109981 -- new job J1 in M1
PDT 2013/07/31 16:43:05.109987 -- starting job J1
PDT 2013/07/31 16:43:05.109996 -- new worker type 100000, min:1, max:1, cmd:`/remote/t3dev1/geis
zler/cdsltest/cdpl/cdsl/bin/rediss'
PDT 2013/07/31 16:43:05.110003 -- new bin B1 in J1
PDT 2013/07/31 16:43:05.110258 -- started worker W1 (as C1) in bin B1
PDT 2013/07/31 16:43:05.233968 -- W1 is `varuna010.internal.synopsys.com:49412'
PDT 2013/07/31 16:43:05.233976 -- W1 log file is /remote/t3dev1/geiszler/cdsltest/cdpl/cdsl/unit
/ds_basic/worker.W1.varuna010.20518.log
PDT 2013/07/31 16:43:05.234989 -- storage server is at varuna010.internal.synopsys.com:60873
```

Sample Server Log with Errors

Here is an example of the log output from log2tty showing a server log from a DP run that had errors when starting the data store server. In this example, the error occurred because the In-Memory Data Store server executable failed to start and register with the application. When such error occurs, check whether the path to the Rediss executable file in the error message is correct.

```
sh-3.00$ log2tty master.*.log
PDT 2013/07/31 17:07:20.249419 -- time: 2013/07/31 17:07:20 PDT -0700
PDT 2013/07/31 17:07:20.249425 -- version: CDPL_I-2013.09_PROD
PDT 2013/07/31 17:07:20.249429 -- system: Linux 2.6.9-67.ELsmp #1 SMP Wed Nov 7 13:56:44 EST 2007
PDT 2013/07/31 17:07:20.249432 -- processor: x86_64 64-bits
PDT 2013/07/31 17:07:20.249434 -- hostname: t3amd03
PDT 2013/07/31 17:07:20.249451 -- new server S952051458
PDT 2013/07/31 17:07:20.249956 -- new master M1
PDT 2013/07/31 17:07:20.249967 -- bcast dir is `/remote/t3dev1/geiszler/cdsltest/cdpl/cdsl/unit/ds_
basic'
PDT 2013/07/31 17:07:20.252875 -- creating SH farm on localhost
PDT 2013/07/31 17:07:20.252893 -- exporting CDPL_LOGDIR as `/remote/t3dev1/geiszler/cdsltest/cdpl/c
dsl/unit/ds_basic'
PDT 2013/07/31 17:07:20.252897 -- exporting CDPL_HOME as `/remote/t3dev1/geiszler/cdpltest/cdpl'
PDT 2013/07/31 17:07:20.252901 -- exporting SNPSLMD_LICENSE_FILE as `26585@us01-lic10:26585@us01-li
c12'
PDT 2013/07/31 17:07:20.252904 -- exporting DISPLAY as `:1.0'
PDT 2013/07/31 17:07:20.252909 -- exporting CDSL_WORKDIR as `/remote/t3dev1/geiszler/cdsltest/cdpl/
cdsl/unit/ds_basic'
PDT 2013/07/31 17:07:20.252917 -- new job J1 in M1
PDT 2013/07/31 17:07:20.252924 -- starting job J1
PDT 2013/07/31 17:07:20.252932 -- new worker type 100000, min:1, max:1, cmd:`/remote/t3dev1/geiszle
r/cdsltest/cdpl/cdsl/bin/rediss'
PDT 2013/07/31 17:07:20.252938 -- new bin B1 in J1
PDT 2013/07/31 17:07:20.253030 -- started worker W1 (as C1) in bin B1
PDT 2013/07/31 17:09:20.268540 -- timed out
PDT 2013/07/31 17:09:20.269793 -- cdplPrelaunch failed - Operation timed out
PDT 2013/07/31 17:09:20.269806 -- Error: failed to start Redis server "/remote/t3dev1/geiszler/cdsl
test/cdpl/cdsl/bin/rediss" - Operation timed out
PDT 2013/07/31 17:09:20.269820 -- _init_redis_server(): Data store initialization error
```

DPCHECK – Infrastructure Validation for the Data Store Server

dpcheck is a utility script used to ensure that the RSH or SSH infrastructure is capable of starting Data Store servers. Run the 'dpcheck -h' command to get the correct usage.

Here is the output of the 'dpcheck -h' command.

% **dpcheck -h**

Usage:

```
dpcheck -h|--help
dpcheck -m {rsh|sge|lsf|ssh|pbs|rtda|nb|slurm}
dpcheck -host {host file}
```

CDPL needs rsh|ssh|lsf|sge|pbs|rtda|nb|slurm to be set correctly for launching workers.

This program allows you to validate the transport settings.

The following checks are performed with the options below.

-m:

```
rsh|ssh: check if localhost login is allowed (passwordless)
sge|lsf|pbs|rtda|nb|slurm: check for environment settings, submit host and
default
queues
automount check on localhost.
```

-host:

This checks the availability of the hosts listed in the host file.
Also, all the checks done with the -m option are performed here.

-i hostlist.txt

Generates a CDPL host file from an input text file containing a list of hosts. The mode must be set to rsh, ssh or sh to use this option. The input file hostlist.txt contains 2 columns. The first column is either a 1 or 0 indicating if the host is enabled or disabled. The second column is the name of the host.

-o hostfile

The -o option is an optional parameter used to specify the name of the CDPL host file that is generated from an input host list. The host list is the input file provided with the -i option. When an input host list is provided, but the -o option is not used, the host data is output to the default file name "hostname.mode.hosts". An exit code of 0 signifies the proper environment.
stdout contains additional information. All errors are reported to stderr.

Checking a Host Information File with dpcheck

To check if your host information file configuration is correct, use the command `dpcheck -host <hostfile>`. In the following example, dpcheck is used to check the configuration of an RSH host information file.

```
% dpcheck -host hosts.rsh
automount process is running at localhost.
Valid hostfile found for DP run.
Valid [rsh] environment found for DP runs.
    : host03: Fri Nov 5 10:20:33 PDT 2010
    : host01 is alive
    : host02 is alive
    : host03 is alive
    : host01: Fri Nov 5 10:20:33 PDT 2010
    : host02: Fri Nov 5 10:20:34 PDT 2010
    : host03: Fri Nov 5 10:20:34 PDT 2010
```

Many simple errors can be detected using this utility.